

C++第三章

1.Functions

- C++ 中可以使用的函数来源：

C++ Standard Library

- **库 (Library)**：是函数和对象的集合。没有库，编程语言就无法执行很多任务。
- **C++ 标准库**：C++ 提供了一整套标准库，确保了代码的可用性和高性能。
(例如：`std::cout`, `std::vector`, `std::sort` 等都是标准库提供的函数或对象。)

User Defined Functions (UDF)

- 如果标准库中没有你需要的功能，就需要自己编写函数，这些叫做 **用户自定义函数**。
- 用户自定义函数可以扩展语言的功能，满足特定场景的需求。
- Declare function in C++ (函数声明方式)

命名函数 (Named function)

- 有输入参数和返回值类型。
- 例如：

```
1 #include <cmath>
2 double sqrt(double);
```

这里声明了一个返回 `double` 的 `sqrt` 函数。

函数对象 (Function object)

- 用输入和返回类型来描述一类函数，而不是具体定义。
- 可以用来表示像 `sin`, `exp` 这种数学函数。
- 例子：

```
1 #include <functional>
2 function<double(double)>;
```

λ (lambda) 函数 (匿名函数)

- 没有名字的小函数，通常用于临时计算。
- 示例：

```
1 auto f = [](double x) { return x * x; };
```

- 表示一个计算平方的函数。

- **How to create a C++ named function (如何创建命名函数)**

C++ 中定义函数有两种方法：

1. 在同一个地方写声明和函数体

```
1 void f(int a) {  
2     cout << a << endl;  
3 }
```

2. 将声明和定义分开 (便于多个 .cpp 文件共享)

- 在头文件 (.h) 里写函数声明
- 在源文件 (.cpp) 里写函数定义

注意:

C++ 标准禁止 **只靠返回类型不同** 来区分函数。

```
1 void f(int a);  
2 int f(int a); // 错误, 只是返回类型不同
```

函数重载 (Overloading) :

允许通过 **输入参数不同** 来定义多个同名函数。

```
1 void f(int a); // OK  
2 void f(string a); // OK, 不同参数类型
```

• Function definition (函数定义)

- 函数定义由两部分组成: **声明 (declaration)** 和 **函数体 (body)**。
- **函数声明**包含: 返回类型、函数名、输入参数 (类型 + 名称)。
- **函数体**包含执行任务的 C++ 语句。
- 一般形式:

```
1 return_type function_name(type1 name1, type2 name2, ...)  
2 {  
3     statements;  
4 }
```

- 要点:

1. `return_type` 决定函数返回值类型, 如果不是 `void`, 必须通过 `return` 返回一个对应类型的值。
2. 如果函数没有返回值, 使用 `void`。
3. 定义后, 函数可以在其它 .cpp 文件中调用 (前提是包含声明), 所以函数声明通常放在头文件 (.h) 里。

Function - separate header and definition (分离函数声明与定义)

- **函数头文件** (header) 只包含声明, 没有函数体, 以分号结尾:

```
1 return_type function_name(parameters);
```

- 编译器只需要知道: 返回类型、函数名、输入参数类型 (以及可选的参数名), 就能在调用时处理函数。
- **定义**在 .cpp 文件中实现:

```

1 void print_hello_world(string s);
2 void print_hello_world(string s) {
3     cout << s << '\n';
4 }

```

◦ 通常我们 **复制声明** 到头文件和实现文件，以保持一致性。

- **Function in single-file case (单文件中的函数)**

◦ 在单文件程序中，可以先在 `main` 函数之前 **声明函数**，然后在 `main` 函数之后 **定义函数**。

◦ 示例:

```

1 #include <iostream>
2 using namespace std;
3
4 void PrintHelloWorld(string); // 函数声明
5
6 int main() {
7     PrintHelloWorld("Hello world!"); // 调用函数
8     return 0;
9 }
10
11 void PrintHelloWorld(string s) { // 函数定义
12     cout << s << '\n';
13 }

```

- **Function in multi-file case (多文件中的函数)**

◦ 在实际项目中，通常把函数的 **声明放在头文件 (.h)**，**定义放在实现文件 (.cpp)**，然后在 `main.cpp` 中通过 `#include` 使用。

◦ 示例:

头文件: print_hello_world.h

```

1 #pragma once
2 void PrintHelloWorld(std::string);

```

实现文件: print_hello_world.cpp

```

1 #include <iostream>
2 #include "print_hello_world.h"
3
4 void PrintHelloWorld(std::string s) {
5     std::cout << s << '\n';
6 }

```

主程序: main.cpp

```

1 #include <iostream>
2 #include "print_hello_world.h"
3
4 int main() {
5     PrintHelloWorld("Hello world!");
6     return 0;
7 }

```

- 编译时需要把多个 `.cpp` 文件一起编译：

```
1 g++ main.cpp print_hello_world.cpp -o main
```

Guidelines for header file (头文件编写规范)

- 不要在头文件中声明变量（否则每次 `#include` 都会生成一个新变量，导致重复定义问题）。
- 应该在头文件中声明：

- 函数
- `struct`
- `const` 变量

- What if we define a function in header file?

- 如果我们在头文件（`.h` 文件）中直接 **定义函数**（而不是仅声明），当该头文件被多个 `.cpp` 文件包含时，就会导致 **函数重复定义**（**redefinition error**）。
- 示例：

a_func.h

```
1 int a_func() {  
2     return 1;  
3 }
```

b_func.h

```
1 #include "a_func.h"  
2 int b_func() {  
3     return a_func() + 3;  
4 }
```

main.cpp

```
1 #include "a_func.h"  
2 #include "b_func.h"  
3  
4 int main() {  
5     return a_func() + b_func();  
6 }
```

🔴 编译时报错：`a_func` 被定义了多次（因为它既在 `a_func.h` 中定义，又在 `b_func.h` 里被间接包含，最后 `main.cpp` 又包含了一遍）。

- #include and #pragma once**

- `#include` 的作用是 **把另一个文件的内容复制到当前文件**。
- 如果头文件被多次包含，就会出现 **重复定义** 的问题。
- `#pragma once` 可以保证头文件只会被编译器包含一次，避免重复包含。

图里展示了：

- `main.cpp` 包含了多个头文件。

- 其中 `hello.h` 又包含了 `iostream`。
- 如果没有 `#pragma once`，`iostream` 的内容可能被包含多次，导致重复定义。

Use of #pragma once

- 正确做法：在每个头文件开头加上 `#pragma once`，确保该文件只会被包含一次。

```
1 #pragma once
2 // header file content
```

- 在早期，C++ 习惯使用 **include guard**，写法如下：

```
1 #ifndef __THIS_HEADER__
2 #define __THIS_HEADER__
3
4 // header file content
5
6 #endif
```

- 现在推荐直接用 `#pragma once`，更简洁且不容易出错。

• Function body

- 函数体定义了作用域，函数体中的所有变量都是 **局部变量**，函数结束后会被销毁。
- 示例代码：

```
1 double add_up_vector(const vector<double> v) {
2     double sum = 0;
3     for (size_t i = 0; i < v.size(); ++i) {
4         sum += v[i];
5     }
6     return sum;
7 }
8
9 add_up_vector({1.0, 3.0, 2.0});
```

这里的 `sum` 是局部变量，函数返回后会消失。

• Return statement

- `return` 语句立即终止函数执行，并将值返回给调用者。
- 如果函数的返回类型是 `void`，可以不用返回值，但可以使用 `return;` 来提前结束函数。
- 对于其他返回类型，必须保证所有执行路径都有返回值，否则会导致运行时错误 (Segmentation Fault)。

示例（错误写法）：

```
1 int test_two(int a, int b) { a + b; } // 没有 return
2 auto x = test_two(1, 2); // 运行时出错
```

Return statement - example

- 如果函数的返回路径不完整，会出错。
- 错误示例：


```

1 int search() {
2     for (size_t i = 0; i < n; ++i) {
3         if (a[i] == x) {
4             return i;    // 当没找到时就没有 return
5         }
6     }
7 }

```

正确示例:

```

1 int search() {
2     for (size_t i = 0; i < n; ++i) {
3         if (a[i] == x) {
4             return i;
5         }
6     }
7     return -1;    // 当没找到时返回 -1
8 }

```

Return multiple values

- C++ 函数一次只能返回一个值，但可以通过 `struct` 或 `tuple` 来返回多个值。

示例:

```

1 struct Point {
2     double x;
3     double y;
4 };
5
6 Point get_point() {
7     return {1.0, 2.0};    // 使用 {} 返回结构体
8 }

```

这样可以在一个函数中返回多个相关值。

2. More on Functions - Part 1

• Function default parameters

这一节介绍了 **默认参数**:

- 有时函数的一些参数可以有默认值，这样调用函数时就不用每次都写全。
- 默认参数可以减少我们在调用函数时输入的参数数量。
- 默认参数必须放在参数列表的 **末尾**。

示例:

```

1 void mean(vector<double> data, bool sample=true);

```

👉 调用时如果没有给 `sample` 传值，就会用默认值 `true`。

Default parameters examples - 1

讲解了在不同场景下如何写默认参数:

1. 如果函数定义写在一个地方

```
1 void mean(vector<double> data, bool sample=true) {  
2     ...  
3 }
```

2. 如果函数头和函数体分离（声明在头文件，定义在源文件）

- 默认值只能写在 **头文件** 里，不能重复写在函数体定义里。

```
1 // in header file  
2 void mean(vector<double> data, bool sample=true);  
3  
4 // in source file  
5 void mean(vector<double> data, bool sample) {  
6     ...  
7 }
```

这是 C++ 的规则：默认参数只能在函数声明处定义一次。

Default parameters examples – 2

- 规则：默认参数必须写在 **没有默认值的参数之后**。
- 示例：

```
1 int Add(int a = 0, int b) {  
2     return a + b;  
3 }
```

a 有默认值 0，而 b 没有默认值。

Use of default parameters

讲述了默认参数在函数体系中的正确使用方式：

- 当一系列函数共享一些参数时，可以用默认参数减少重复输入，但只能在 **最外层接口** 使用。
- 默认参数不能替代函数之间的 **结构化绑定 (structural binding)**。
- 如果要改默认值，必须记得所有用到默认值的函数都要修改。

示例：

```
1 Cashflow create_cashflow(double maturity=5.0, double coupon_rate=0.0,  
   double face_value=100.0);  
2  
3 Bond create_bond(double maturity=5.0, double coupon_rate=0.0, double  
   face_value=100.0) {  
4     Cashflow cf = create_cashflow(maturity, coupon_rate, face_value);  
5 }
```

默认参数放在 `create_bond` 里更合理，而不是 `create_cashflow`。

• Function overloading

介绍了 **函数重载**：

- 函数名相同，但 **输入参数不同**，可以构成重载。
- “不同”指的是 **参数类型不同** 或 **参数个数不同**。

- 注意：返回值不同或默认参数不同 **不能构成函数重载**。

示例：

```
1 int f(int a, int b) { return a; }; // OK
2 int f(int b, int a) { return a; }; // ✗ 歧义，类型一样只是名字不同
3 int f(int a, string b) { return a; }; // ✓ 输入参数类型不同，可以重载
```

函数重载 (Function Overloading)：允许同名函数根据不同参数类型/数量有不同实现。

例子：

```
1 void print(double x) { cout << "double: " << x << '\n'; }
2 void print(float x) { cout << "float: " << x << '\n'; }
3 void print(int x) { cout << "int: " << x << '\n'; }
4 void print(int x, bool b) { cout << "int and bool: " << x << '\n'; }
```

测试时：

- `print(1.0);` 调用 double 版本 ✓
 - `print(1.0f);` 调用 float 版本 ✓
 - `print(1, true);` 调用 (int, bool) 版本 ✓
 - `print(1);` 报错 ✗，因为 `print(int)` 和 `print(int, bool=false)` 冲突，编译器不确定调用哪个。
- **Use auto in function**
 - C++ 可以用 `auto` 来推断函数返回值。
 - 例子：

```
1 void print_vector(vector<int> vs) {
2     for (size_t i = 0; i < vs.size(); ++i)
3         cout << vs[i] << ", ";
4 }
5
6 // 等价写法
7 auto print_vector(vector<int> vs) { ... }
```

- 注意：函数参数**不能**用 `auto`（除非用模板 `template`）。

- **函数调用机制**

展示了 **函数调用时内存中的数据流动**。

调用过程：

1. `main` 中的实参（`a=1, b=2`）被复制到函数的形参内存区域。
2. 函数在自己的作用域里运行（生成局部变量，如 `res`）。
3. 当 `return` 时，结果返回到调用者作用域。
4. 函数结束，局部变量被释放。

进一步解释 **函数调用机制**：

1. 输入参数被复制到函数的内存区（传值）。
2. 函数可以定义新的局部变量。

3. 返回时，基本类型按值复制回去；对象会用 move（更高效）。

4. 函数结束后，局部变量作用域结束，被销毁。

- 函数参数传递方式（值传递 vs 引用传递）

- 函数参数在函数体内被当作 **局部变量** 创建。
- 函数体中定义的所有变量也都是局部变量。
- 当函数返回时，所有局部变量会被销毁。
- 如果希望在多次函数调用之间保存状态，可以用 **static 静态变量**（后续会介绍）。

Pass by value（值传递）

- 默认情况下，函数参数是通过 **值传递** 的，即调用时会把实参的值复制给形参。
- **影响：**
 1. 函数内部修改形参，不会影响调用者的实参。
 2. 对于大对象（如大数组、vector），值传递会导致额外的复制，效率低下，占用更多内存。

Reference type（引用类型）

- 为了解决值传递的问题（无法修改实参、效率低），C++ 提供了 **引用传递（Pass by Reference）**。
- **引用（reference）** 是一个变量的别名，本质上保存的是变量的地址。
- 当函数参数是引用时，函数直接操作调用者的实参，而不是副本。
- 引用的大小固定（64 位系统上是 8 个字节），与其指向的数据大小无关。

Reference - syntax（语法）

1. **创建引用：**在变量名前加 **&**。

```
1 int x = 10;
2 int &r = x; // r 是 x 的引用
```

2. **引用必须初始化**，且一旦绑定，不能再引用其他变量。

例如：

```
1 int x = 5, y = 10;
2 int &r = x; // r 引用 x
3 r = y; // 改变的是 x 的值，而不是让 r 改变引用对象
```

3. **const 引用：**

可以防止通过引用修改值。

```
1 const int &r = x; // 不能通过 r 修改 x
```

4. `const T&` 和 `T const&` 含义相同，表示对一个常量值的引用（不能修改）。

- C++ 引用（reference）的使用和示例

Reference - example 1

- **定义引用变量：**

```
1 int x{3}, y{4};
2 int &r1 = x;    // 创建引用 r1, 绑定到 x
```

- 引用必须初始化:

`int &r1;` 会报错, 因为引用必须立即绑定。

- 引用不可重新绑定:

`r1 = y;` 不会让 `r1` 引用 `y`, 而是把 `y` 的值赋给 `x`。

- `const` 引用:

可以用 `const int&` 来绑定变量, 防止修改。

- `auto` & 引用:

对于大对象 (如 `vector`), 可以用 `auto&` 来避免拷贝, 提高效率。

Reference - example 2

- 在函数参数前加 `&`, 即可传引用, 而不是传值。

- `const` & 用法: 常用来避免不必要的拷贝, 并防止修改传入的参数。

- 建议:

- 对于 `int/double/bool` 等小数据类型, 用 **值传递** 即可;
- 对于 `struct/vector` 等大数据类型, 用 **引用传递** 更高效。

Reference - example 3

- **传值**: `foo(vector<int> v)` → 修改副本, 不影响原始参数。

- **传引用**: `foo2(vector<int>& v)` → 直接修改原始参数。

- **const 引用**: `foo2(const vector<int>& v)` → 不能修改原始参数, 保证安全。

Reference - example 4

- 演示了 **引用传递与值传递的区别**:

- `update1(int v)` 是值传递, 只能修改副本。
- `update2(int& v)` 是引用传递, 能修改原始值。

- 注意: 函数不能对字面量 (如 `1`) 或临时值 (如 `{1, 2, 3}`) 使用非常量引用。

- 可以用 **const 引用** 来绑定临时值或字面量。

- 内联函数优化 **Potential optimization from inline**

- 示例:

```
1 inline int add(int a, int b) { return a+b; }
2 int c=5, d=10;
3 cout << add(c,d) << '\n';
```

- 解释:

- 函数调用需要有数据拷贝和压栈过程, 如果函数体非常简单, 开销可能比函数本身更大。
- `inline` 建议编译器将函数调用替换为函数体代码本身 (直接在调用点展开)。
- 这样避免了函数调用的开销, 提升效率。
- 注意: `inline` 是一种 **提示**, 编译器可以选择是否真正展开。

- **Return local variables**

- 在 C++11 之前，如果函数里定义局部大对象（如 `vector`），并返回，通常会担心拷贝开销，常用指针或引用返回。
- 现代 C++（C++11 起）有 **move 语义**，直接返回局部大对象是安全且高效的：
 - 编译器会自动调用 **移动构造函数** 而不是拷贝构造函数。
- 示例：

```

1  vector<int> return_vector(int n) {
2      vector<int> vec(n, 5);
3      cout << &vec[3] << ", " << vec[3] << '\n';
4      return vec; // 会发生移动，而不是拷贝
5  }
6
7  int main() {
8      auto x = return_vector(10);
9      cout << "in main()" << '\n';
10     cout << &x[3] << ", " << x[3] << '\n';
11 }

```

- 输出显示：返回后的 `x` 与函数中的 `vec` 实际上接管了相同的数据存储，避免了拷贝。

Return local variables - explanation

- C++11 引入的 **move 操作** 解决了返回局部变量效率低的问题。
- move** 的行为：
 - 对 **简单类型**（`int`, `bool`, `double`）：**move** 实际就是拷贝。
 - 对 **复杂类型**（如 `vector`, `string`）：**move** 会转移内部资源（例如动态数组的指针），原对象置为空。
- 就像“资金转账”：数据所有权转移，不产生新的拷贝。
- 示例 1（基础类型）：

```

1  int x = 3;
2  int y = move(x);
3  cout << x << ", " << y << '\n'; // 输出 3, 3

```

- 基本类型没有资源可转移，所以 **move** 等同于拷贝。
- 示例 2（容器类型）：

```

1  vector<int> x1{3, 3};
2  auto x2 = move(x1);
3  cout << x2.size() << ", " << x1.size() << '\n'; // 输出 2, 0

```

- `x2` 接管了 `x1` 的资源，`x1` 被清空。
- 函数返回多个值的方法

在 C++ 中，函数只能返回一个值。要返回多个值，可以通过：

- 使用 **引用参数（reference）**：把多个返回值作为函数输入参数的引用传入，但这样会导致函数参数表太长，不够直观。
- 使用 **struct**：定义一个结构体来存储多个值，适合需要保存“记录型”数据的场景。
- 使用 **tuple**：更简单，支持不同类型的数据打包在一起。

- Tuple

- tuple 是 C++ 提供了一种 **固定大小的异构集合**，一旦创建后不可变。
- 需要包含头文件：

```
1 #include <tuple>
```

- 声明和初始化：

```
1 tuple<int, double, string> t = {1, 3.0, "abc"};  
2 auto t = make_tuple(1, 3.0, string("abc"));
```

- 访问元素的方式：

1. 解包赋值 (C++17 起可用)：

```
1 auto [v1, v2, v3] = t;
```

早期版本需要 tie：

```
1 tie(v1, v2, v3) = t;
```

2. 使用 get 按索引访问：

```
1 auto a = get<0>(t);  
2 auto b = get<1>(t);  
3 auto c = get<2>(t);
```

Tuple - example

- 函数返回 tuple：

```
1 tuple<int, double, string> get_tuple() {  
2     return make_tuple(1, 2.3, "456"s);  
3 }  
4  
5 int main() {  
6     auto [x, y, z] = get_tuple();  
7     cout << x << ", " << y << ", " << z << '\n';  
8 }
```

- 输出：1, 2.3, 456
- 可以直接解包多个返回值。

- 另一种写法 (老式解包方法)：

```
1 int x; double y; string z;  
2 tie(x, y, z) = get_tuple();
```

- 或者使用 get 获取单个元素：

```

1 auto t = get_tuple();
2 auto a = get<0>(t);
3 auto b = get<1>(t);
4 auto c = get<2>(t);

```

• Summary – return values from function

函数返回值的总结方式有两种：

1. 传引用参数 (pass by reference) :

- 用 & 传入参数，使得函数内部的修改会影响调用者的变量。
- 例子中 `update(int &v)` 让 `x` 自增。
- `update(vector<int> &v)` 修改了 `vector` 的大小，并且更新了某个元素。

2. 在函数内部创建局部变量 (如 `vector`、`tuple`、`struct`) 并返回：

- 示例用 `struct Point` 定义一个点结构体，并返回 `vector<Point>`。

• Counting time in C++11

- 使用 `<chrono>` 库测量程序执行时间。
- 代码示例：
 - 使用 `high_resolution_clock::now()` 获取当前时间点。
 - 使用 `duration_cast<nanoseconds>` 计算时间差。
 - 例子中运行一个循环 (调用 `test_value_copy` 100 万次)，最后输出所耗时间。
- 这是现代 C++ 提供的高精度计时方式。

示例：

```

1 #include <chrono>
2 #include <iostream>
3 #include <vector>
4 using namespace std;
5 using namespace std::chrono;
6
7 void test_value_copy(vector<int> x) { return; }
8
9 int main()
10 {
11     vector<int> xx(1'000'000ULL, 1);
12     auto start = high_resolution_clock::now();
13     for (size_t i = 0; i < 1'000'000ULL; ++i) {
14         test_value_copy(xx);
15     }
16     double time_taken = duration_cast<nanoseconds>(
17         high_resolution_clock::now() - start).count() / 1e9;
18     cout << time_taken << '\n';
19 }

```

• Static Variable in a Function

- 静态变量 (`static`) 在函数内的特点：
 - 与普通局部变量不同，`static` 变量存在于独立的存储区，函数返回后不会被销毁。
 - 只会初始化一次，以后函数调用时继续使用之前的值。

- 示例 `keep_track()` 里定义了 `static int count`：

```
1 keep_track(); // 输出 1
2 keep_track(); // 输出 2
```

- 用途：在某些问题中快速保存状态，但要谨慎使用，因为静态存储空间有限。

• Generate a random number in C++11

- 使用 `<random>` 库生成随机数 (C++11 新特性)。

- 示例：

```
1 random_device rd{};
2 auto mtgen = mt19937(rd()); // 随机数引擎
3 auto urdd = uniform_real_distribution<>(2.0, 5.0); // 区间 [2,5] 的均匀分布
```

- 每次运行会得到不同的随机数。
- `rd` 和 `mtgen` 建议作为全局变量存在 (生成器本身与外界无关)。
- 也可以用 `normal_distribution` 生成正态分布随机数。

```
1 #include <iostream>
2 #include <random>
3
4 int main() {
5     // 1) 引擎：用 random_device 产生种子
6     std::random_device rd;
7     std::mt19937 mtgen{ rd() }; // 如果想可复现，用 mt19937
8     mtgen{123456u};
9
10    // 2) 分布：连续均匀分布 [2.0, 5.0)
11    std::uniform_real_distribution<double> urdd(2.0, 5.0);
12
13    // 3) 抽样 10 次
14    for (int i = 0; i < 10; ++i) {
15        double x = urdd(mtgen); // 从分布里用引擎取一个样本
16        std::cout << x << "\n";
17    }
```

随机数引擎的“种子”与引擎

```
1 random_device rd{};
2 auto mtgen = mt19937{rd()};
```

- `random_device rd{};`

- 申请一个“随机设备”，用于产生种子。它通常从操作系统/硬件收集熵，能给你“不可预测”的整数。
- 在某些实现/平台上，如果没有真正的硬件熵源，可能会退化为确定性的伪随机源 (依实现而定)。

- `auto mtgen = mt19937{rd()};`

- 创建一个 **梅森旋转 (Mersenne Twister)** 伪随机引擎 `mt19937`，并用上面 `rd()` 产生的整数做种子。
- `mt19937` 速度快、质量高，广泛使用，它本质是**确定性的**，给定同样的种子就会产生同样的序列。
- `auto` 会被推断为 `mt19937` 类型。

小结：`rd` 只负责**一次性**提供“更随机”的种子，真正出随机序列的是 `mtgen`。**引擎要复用**，不要每次都重新创建或重新播种（会变慢且影响统计性质）。

选择一个“分布”（把引擎输出的整数映射成你要的形式）

```
1 void some_func() {
2     // Create a distribution
3     auto urdd = uniform_real_distribution<>{2.0, 5.0};
4     // Or, normal_distribution<> std::norm(0.0, 1.0);
5     ...
6 }
```

- `uniform_real_distribution<>`：**连续均匀分布**，默认模板参数是 `double`，括号里给出区间 `[a, b)`。
 - 这里是 `[2.0, 5.0)`：**包含 2.0，不包含 5.0**（右端开区间，标准规定“结果至少为 `a` 且小于 `b`”）。
 - `auto urdd` 会被推断为 `uniform_real_distribution<double>`。
- 注释里给的备选：`normal_distribution<> std::norm(0.0, 1.0);`
 - 这是**正态分布**，第一个参数是**均值** (0.0)，第二个是**标准差** (1.0)。

引擎只会产生“原始的不均匀整数”，**分布**把它“变形”为你想要的统计分布（均匀、正态、伯努利.....）。

抽样（真正“取一个随机数”）

（要写在 `urdd` 仍在作用域内的地方，例如 `some_func` 里）：

```
1 for (auto i = 0; i < 20; ++i) {
2     auto number = urdd(mtgen);
3     // 使用 number (一个 double, 落在 [2.0, 5.0))
4 }
```

- `urdd(mtgen)` 的语法：**把引擎传给分布**，分布对象重载了 `operator()`，每调用一次就**抽样一次**，返回一个 `double`。
- 循环 20 次，就得到 20 个样本。

• Task-oriented functions and recipe function

- **任务导向函数**：每个函数完成一个单一任务，例如 `make_dough()`、`make_sauce()` 等。
- **配方函数 (recipe function)**：把多个任务函数组合起来，形成一个完整流程，例如：
 - `make_pizza()`：依次调用 `make_dough()`、`make_sauce()`、`make_topping()`、`mix()`、`bake()`，最后返回完整的 `pizza`。
 - `price_bond()`：依次创建 `bond`，计算 `cashflow`，再计算折扣系数，最后得到价格并返回。
- 体现了**模块化编程思想**：小函数负责独立任务，大函数通过调用它们组合实现复杂功能。

• Demo: Line number - 1

- 介绍文本编辑器中的两种**行号模式**:

- 静态模式 (static mode)**: 行号始终是 1, 2, 3, 4, 5, ... (从上到下固定不变)。
- 动态模式 (dynamic mode)**: 行号相对于当前行计算, 例如: 当前行是 4, 则显示为 4, 3, 2, 1, 0, 1, 2, 3, 4, ...。

- 要实现两个函数来模拟这两种模式。

• Demo: Line number - 2

- 说明了如何设计函数接口:
 - 需要复用行号存储变量, 所以使用**引用类型 (&)**传递 `vector<int>`。
 - `current` 表示当前行号, 假定其值合法 (1 到 n 之间)。
- 初始函数定义:

```
1 void line_number_static(vector<int>& line_numbers);  
2 void line_number_dynamic(size_t current, vector<int>& line_numbers);
```

- 统一函数签名 (便于调用时接口一致):

```
1 void line_number_static(size_t current, vector<int>& line_numbers);  
2 void line_number_dynamic(size_t current, vector<int>& line_numbers);
```

• Demo: Line number - 3

- 给出了 `line_number_static` 的一个实现:

```
1 void line_number_static(vector<int>& line_numbers) {  
2     for (size_t i = 0, j = 1; i < line_numbers.size(); ++i, ++j) {  
3         line_numbers[i] = j;  
4     }  
5     return;  
6 }
```

- `line_numbers` 向量被依次填充 1, 2, 3, 4, ..., 实现了静态模式。

- 下面也给出了一个**容易出错的写法**:

```
1 line_numbers[0] = 0;  
2 for (size_t i = 1, j = 0; i < line_numbers.size(); ++i, ++j) {  
3     line_numbers[i] = ++line_numbers[j];  
4 }
```

- 这里 `line_numbers[j]` 在更新过程中可能导致逻辑混乱 (因为使用了之前已经修改过的值)。

• Demo: Line number - 4

实现了**动态模式行号**:

```

1 void line_number_dynamic(int current, vector<int>& line_numbers) {
2     const auto current_1 = current - 1;
3     for (size_t i = 0; i < line_numbers.size(); ++i) {
4         if (i < current_1) {
5             line_numbers[i] = current_1 - i; // 当前行之前: 递减
6         } else if (i == current_1) {
7             line_numbers[i] = current; // 当前行: 显示自身
8         } else {
9             line_numbers[i] = i - current_1; // 当前行之后: 递增
10        }
11    }
12 }

```

作用: 让当前行显示 `current`, 前面逐步递减, 后面逐步递增。

• Demo: Line number - 5

对 `line_number_dynamic` 进行了**优化**:

- 把 `line_numbers.size()` 存到常量 `ln_size`, 避免每次循环重复调用。
- 把 `if` 语句逻辑拆分, 先填充当前行前面的部分, 再填充后面的部分, 减少分支判断。

优化后代码:

```

1 void line_number_dynamic_opt(int current, vector<int>& line_numbers) {
2     const auto current_1 = current - 1;
3     const auto ln_size = line_numbers.size();
4     for (size_t i = 0; i < current_1; ++i) {
5         line_numbers[i] = current_1 - i; // 前半部分
6     }
7     line_numbers[current_1] = current; // 当前行
8     for (size_t i = current_1 + 1; i < ln_size; ++i) {
9         line_numbers[i] = i - current_1; // 后半部分
10    }
11 }

```

优点: 去掉了循环内部的 `if-else`, 更高效。

• Demo: Line number - 6

写了一个**测试函数** `test_line_number`, 用来统一测试三种模式函数 (静态模式、动态模式、优化动态模式)。

关键点:

- 使用了**函数对象** `function<void(int, vector<int>&)>`, 可以把不同的函数作为参数传入。
- 定义了一组 `current_test_cases = {1, 2, 7, 12, 17}`, 逐一测试。

```

1 void test_line_number(string msg, function<void(int, vector<int>&>>
  line_number_gen) {
2     cout << msg << '\n';
3     auto lines_numbers = vector<int>(17, 0);
4     auto current_test_cases = vector<int>{1, 2, 7, 12, 17};
5     for (size_t i = 0; i < current_test_cases.size(); ++i) {
6         cout << "Test: " << current_test_cases[i] << '\n';
7         line_number_gen(current_test_cases[i], lines_numbers);
8         print_vector(lines_numbers);
9     }
10    cout << '\n';
11 }

```

main() 里调用:

```

1 int main() {
2     test_line_number("line_number_static: ", line_number_static);
3     test_line_number("line_number_dynamic: ", line_number_dynamic);
4     test_line_number("line_number_dynamic_opt: ",
  line_number_dynamic_opt);
5 }

```

作用: 一次性比较三种行号生成模式, 输出结果用于验证。

• Useful Function When Working With Numbers

- 讲解了如何比较两个浮点数是否“相等”。
- 因为浮点数计算存在精度问题, 不能直接用 `==` 比较。
- 提供了函数:

```

1 bool is_close(double a, double b, double threshold = 1e-5) {
2     return fabs(a - b) <= threshold;
3 }

```

使用一个阈值 `threshold` 判断两个数是否接近。

- 更严格的版本 `is_close_strict`, 用到了 `std::numeric_limits<double>::epsilon()` 来考虑浮点数的相对误差。